

VulnusLab

Standard Operating Procedure (SOP)

Vulnerability Assessment & Penetration-Testing Platform

Document ID	VL-SOP-001
Version	1.0
Date	2026-06-13
Owner	VulnusLab (MSME UDYAM-AP-13-0090768)
Classification	CONFIDENTIAL
Review cycle	Quarterly

This document is the authoritative operating procedure for building, deploying, operating, and securing the VulnusLab platform. It is confidential and intended for internal operators and authorized auditors only.

1. Purpose & Scope

This SOP defines the standard, repeatable procedures for operating VulnusLab — an automated security-testing platform that performs Vulnerability Assessment (VA) and, where explicitly authorized, Penetration Testing (PT) against customer-owned assets. It is the single source of truth for how the system is built, deployed, run, monitored, backed up, and secured.

Scope

- The production platform: backend API, frontend dashboard, scanner modules, and engines.
- Operational tasks: deployment, scanning, backup/restore, health monitoring, module builds.
- Governance: access control, change management, incident response, and compliance.
- Out of scope: customer engagement contracts and per-customer rules of engagement (handled separately).

Authorized use only

VulnusLab must only be pointed at assets the customer owns or has written authorization to test. The platform is VA-first (detect and report); active exploitation is confined to the single authorized autonomous PT module operating under a non-destructive safety contract.

2. System Overview

VulnusLab is a single-tenant-per-customer SaaS delivered as a containerized application on a VPS, with a browser dashboard. It bundles industry-standard open-source security engines behind a uniform API so a customer can run professional-grade assessments without assembling a toolchain.

Metric	Value
Scan modules	42 (plus a credential-vault service)
Detection routines	~1,572 (574 tier-style scanners + 998 pack probes)
Security engines bundled	51 binaries (nmap, nuclei, trivy, sqlmap, hydra, etc.)
Finding rule-sets	305 findings modules
Curated wordlists	40 lists (~156,000 entries)
Delivery	Docker on VPS; React dashboard at app.vulnuslab.com

Design principles

- **Kali-first, pure-Python fallback:** use the best engine where available; otherwise a self-contained Python probe.
- **Zero false positives:** every finding is actively verified; confidence is labelled (CONFIRMED vs SUSPECTED).
- **VA-not-PT by default:** post-compromise / exploitation techniques are emitted as advisory information, never as fake high-severity findings.
- **Module isolation:** each module owns its scanners, payload pool, orchestrator, and UI wiring.

3. How It Works — Architecture

A scan flows from the dashboard through a per-module orchestrator that fans out to the module's scanners, streams results back live, and renders them into a customer PDF report.

Request flow

- **Frontend** (React SPA, served by nginx): the operator selects a module, supplies the target and any inputs/credentials, and starts the scan.
- **nginx reverse proxy** forwards the request to the backend; streaming endpoints are configured un-buffered so each result reaches the browser immediately.
- **Backend** (FastAPI + Uvicorn, 4 workers): an autoloader registers every scanner at boot (currently ~897 tools, 0 failed) with a self-healing engine.
- **Orchestrator** (one per module): resolves the requested tiers to a tool list and runs them concurrently, emitting newline-delimited JSON (NDJSON) as each finishes.
- **Scanners**: each calls an engine (or a pure-Python probe), normalizes output into findings (severity, evidence, remediation), and returns a uniform response.
- **Report**: the dashboard aggregates, de-duplicates, and renders an industry-standard PDF.

Component layout

Component	Technology	Role
Frontend	React SPA + nginx (Docker)	Dashboard UI, PDF generation, auth
Backend	FastAPI + Uvicorn (Docker)	API, autoloader, orchestrators, scanners
Engines	51 OSS binaries in the image	Actual scanning (shared by all modules)
Datastore	SQLite (users, scans, plans)	Persisted in a Docker volume
Cred vault	Encrypted per-user store	Holds customer cloud/AD/K8s credentials
Lab targets	Vulnerable containers	Internal test targets (DVWA, Juice Shop, k3s, etc.)

4. What You Need — Prerequisites

To stand up and operate a VulnusLab instance, the following are required.

Infrastructure

- **VPS**: Linux (Ubuntu/Debian 22.04+), minimum 4 vCPU / 8 GB RAM / 80 GB disk; swap enabled (the frontend build is memory-intensive).
- **Docker + Docker Compose** installed.
- **Domain + DNS**: an A record (e.g. app.vulnuslab.com) pointing at the VPS; TLS via nginx.
- **Outbound internet** for engine vulnerability databases (OSV, nuclei-templates, CVE feeds).

Configuration & secrets

- **.env file** (never committed): JWT secret, admin password, CORS origins, and API keys (VirusTotal, AbuseIPDB, payment processor when enabled).
- **Engine image**: the Docker image bundles all 51 engines and their databases (~5 GB).
- **Customer credentials**: stored only in the encrypted vault, never in code or .env.

People & access

- At least one super-admin operator account.
- SSH access to the VPS (key-based; root password login disabled — see Security).
- A GitHub repository for code (code only; never customer data or secrets).

5. Standard Operating Procedures

5.1 Deploy / redeploy

Code is committed and pushed from the build environment; the VPS pulls and rebuilds the affected service. Backend and frontend are rebuilt independently.

```
git pull
docker compose build backend # or: frontend
docker compose up -d backend
```

5.2 Run a scan

- Select the module in the dashboard; complete the scan-setup form (target, inputs, credentials).
- Start the scan; results stream live per scanner (status, severity, count, elapsed).
- Review findings, then export the customer PDF report.

5.3 Build or extend a module (VL-FORGE)

New modules are built to 100% of their playbook using the VL-FORGE process: delete any old module code, then forge scanners + payload pool + orchestrator + frontend wiring + PDF section, and validate against the 7-check Definition of Done.

5.4 Quality gate (VL-FOUNDRY)

Every module is scored by the automated scorer across six layers (orchestrator coverage, AI-curation, 7-check quality bar, parallelism, frontend, UI integration). A module must score ≥ 85 to ship; the fleet target is 100. The score measures structural readiness, not real-world coverage — live validation is a separate step (VL-AUDIT).

5.5 Backup & restore

- Automated daily database backup at 03:00 (cron) to /root/backups, 14-day retention.
- Restore by stopping containers, copying the chosen backup over the live volume, and restarting.
- Off-VPS copies (private repo / object storage) are strongly recommended for disaster recovery.

5.6 Health monitoring

The System Health view and /api/health report backend status, scanner load count, failed/healed scanners, and the healing-engine state. Container health is verified with `docker compose ps`.

6. Quality Framework — VL Processes

VulnusLab uses a set of named, repeatable engineering processes. The first group is the build-to-ship pipeline applied to every module; the rest are selective enhancers.

Process	Type	Purpose
VL-FORGE	Universal	Build a module to 100% of its playbook
VL-FOUNDRY	Universal	Automated 6-layer readiness scoring
VL-FLOW	Universal	Frontend phase / sidebar wiring
VL-STREAM	Universal	Live NDJSON result streaming
VL-CORE	Selective	Per-module isolated payload pool
VL-VERIFY	Selective	SPA-canary false-positive suppression
VL-METHOD	Selective	7-step methodology for multi-step workflows
VL-AUDIT	Platform	One-command live scan of every module
VL-DEDUP	Platform	Cross-module finding de-duplication
VL-PRIME	Platform	Encrypted master archive (vault)

7. Security & Compliance

As a security product handling sensitive customer data and powerful tooling, VulnusLab operations must meet the controls below. Items marked (gap) are recommended additions to reach industry standard.

7.1 Access control

- Role-based accounts (super-admin, admin, operator); least privilege enforced.
- JWT-authenticated API; per-user scoping of scans and credentials.
- SSH to the VPS is key-based only; root password authentication disabled.

7.2 Secrets management

- Secrets live only in .env (gitignored) and the encrypted credential vault — never in code.
- Customer credentials are encrypted per-user; plaintext never leaves the backend process.
- **Lesson learned:** a plaintext credential note was once committed to git history; affected credentials must be rotated and history treated as exposed. No secrets in the repo, ever.
- **(gap)** Adopt a managed secrets store and automated secret-scanning in CI.

7.3 Authorization of testing

- Only scan assets the customer owns or is authorized to test; capture authorization per engagement.
- **(gap)** Enforce a signed scope/consent gate before high-impact modules can run.

7.4 Data protection & retention

- Scan results and reports are customer data; store the minimum necessary and define retention.
- TLS in transit (nginx); database on an encrypted-at-rest volume is recommended.
- **(gap)** Document a data-retention schedule and a customer data-deletion procedure (GDPR/DPDP).

7.5 Audit logging

- Authentication, scan starts, and credential use are recorded.
- **(gap)** Centralize tamper-evident audit logs and alert on anomalies.

8. Roles & Responsibilities

Role	Responsibilities
Platform Owner	Strategy, prioritization, compliance accountability, approvals
Operator / Analyst	Runs scans, reviews findings, produces customer reports
Engineer	Builds and maintains modules, deploys, fixes defects
Security Officer	Owens access control, incident response, secret hygiene
Customer	Provides authorization and scope; owns the tested assets

9. Change Management

- All changes flow through version control; commits are descriptive and attributable.
- Build in a non-production context, validate (compile + VL-FOUNDRY + live spot-check), then deploy.

- Backend and frontend are deployed independently to limit blast radius.
- Maintain a last-known-good snapshot and a documented rollback (revert + redeploy).
- (gap)** Use pull-request review and a staging environment before production.

10. Incident Response

Triggers include: backend outage, credential exposure, scanner mass-failure, or abuse of the platform.

Phase	Action
Detect	Health checks, error logs, customer report
Contain	Isolate the affected service; revoke/rotate exposed credentials
Eradicate	Fix root cause; redeploy from known-good
Recover	Restore data from backup; verify health
Review	Post-incident write-up; update this SOP and add a regression check

11. Disaster Recovery & Continuity

- Code:** recoverable from the Git remote at any time.
- Data:** daily DB backups (14-day retention) + recommended off-VPS copies.
- Full rebuild:** provision a new VPS, install Docker, clone the repo, restore .env and the latest database backup, deploy, then repoint DNS.
- (gap)** Define RPO/RTO targets and rehearse the recovery at least annually.

12. Compliance Mapping

The scanning content aligns to widely-recognized standards. Operational compliance (of running the business) has documented gaps to close for enterprise sales.

Standard	Relevance	Status
OWASP / MASVS / API Top 10	Scan coverage frameworks	Implemented
NIST SP 800-115 / 800-218	Testing & secure-SDLC method	Aligned
SLSA, Sigstore, EU CRA, EO 14028	Supply-chain assurance	Partial
SOC 2 / ISO 27001	Operational security of the SaaS	(gap) recommended
GDPR / India DPDP	Customer data handling	(gap) to formalize

13. Recommended Additions (to reach industry standard)

Consolidated list of the (gap) items above, in priority order:

- Rotate any historically-exposed credentials; add CI secret-scanning.

- Add a signed scope/authorization gate before high-impact scans.
- Formalize data-retention and customer data-deletion procedures (GDPR / DPDP).
- Centralize tamper-evident audit logging with alerting.
- Introduce pull-request review + a staging environment.
- Define RPO/RTO and rehearse disaster recovery annually.
- Pursue SOC 2 / ISO 27001 readiness for enterprise customers.
- Add off-VPS encrypted backups as the default, not optional.

End of document. Review quarterly or after any material architecture or process change. This SOP is maintained in version control alongside the platform.